VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software

Student Name	Registration Number
Ekansh Jain	24BCE11500
Slot	**Date of Submission**
E22	29/03/2026
License	**Software**
GPL v2	Linux Kernel

1. Introduction

The Linux kernel is a free and open-source, Unix-like kernel that is used in many computer systems worldwide. The kernel was created by Linus Torvalds in 1991 and was soon adopted as the kernel for the GNU operating system (OS) which was created to be a free replacement for Unix. This audit examines the Linux Kernel, the foundational software that bridges hardware and software, powering everything from Android phones to the world's most advanced computational physics clusters. Unlike high-level application programming, which can sometimes feel repetitive or abstracted, the kernel requires a rigorous understanding of system architecture, memory mathematics, and hardware physics.

2. Project Report Structure

Part A — Origin and Philosophy.

A1. The problem this software was created to solve

Linus Torvalds was a student who wanted a free Unix-like system and could not afford one. He wanted to access his university's Unix servers from his 386 PC and found the existing options too expensive or limited.

A2. The license — what it actually says

The Linux Kernel is licensed under the GNU General Public License Version 2(GPL v2)

The free software are defined by Free Software Foundation through four fundamental freedoms

Freedom 0: The freedom to run the program as you wish, for any purpose.

Freedom 1: The freedom to study how the program works and change it so it does your computing as you wish. Access to the source code is a precondition for this.

Freedom 2: The freedom to redistribute copies so you can help others.

Freedom 3: The freedom to distribute copies of your modified versions to others.

The GPL v2 grants all four of these freedoms, ensuring that the Linux Kernel remains a community asset.

A company cannot take the Linux Kernel, modify it, and sell it without sharing their changes as the GNU v2 is a copyleft license meaning that if a company modifies the Linux Kernel and distributes that compiled software, the GPL v2 legally requires them to release the source code of their modifications under the exact same license. This mechanism prevents corporate lock-in and ensures that any improvements made to the kernel are fed back into the public ecosystem. (However, if a company modifies the kernel solely for internal server use and never distributes the binaries, they are not forced to publish their code, a loophole heavily utilized by cloud providers today).

The MIT License is highly permissive as it allows anyone to take the code, modify it, and distribute it as closed-source proprietary software. If Torvalds had chosen the MIT license, massive corporations could have taken the Linux code, optimized it for their hardware, and sold it without ever contributing back. The ecosystem would have fragmented. I would have chosen GPL License because GPL v2, by contrast, enforces an infectious freedom. If you distribute GPL code, your distribution must also be GPL. For a foundational infrastructure project like a kernel, GPL is the superior choice because it forces commercial beneficiaries

to participate in the open-source social contract. If I were releasing a lightweight JavaScript library, I might use MIT to encourage maximum adoption. But for system-level architecture, the GPL protects the community's labor from exploitation.

"Free beer" refers to software that costs zero money, while "free freedom" refers to software that respects user rights. Freeware like Discord or Adobe Acrobat Reader is "free beer" as you don't pay for it, but you cannot see the code, modify it, or understand what it does with your data. The Linux Kernel is "free as in freedom." The fact that it is also zero-cost is secondary to the fact that developers have the ultimate right to audit, fork, and control the code.

A3. The ethics of open source

Open source is not just a licensing model, it is a statement about how knowledge and tools should be shared.

The argument for open-sourcing all software hinges on transparency, security, and the democratization of knowledge. Critical infrastructure such as the algorithms routing power grids, managing banking data, or analyzing medical diagnostics should theoretically be open for public audit. Hidden code in infrastructure is a societal vulnerability.

Conversely, the argument against universal open source lies in economic incentive and niche innovation. Developing highly specialized software (like industry-specific DAWs for EDM music production or complex CAD software) requires massive upfront capital. Without the ability to sell proprietary licenses, companies might lack the financial incentive to take high-risk technological leaps.

The central tension of modern open source, Trillion-dollar companies run their cloud empires on Linux. On one hand, the GPL v2 allows this, and these corporations often employ core kernel maintainers and contribute massive amounts of code back upstream. In this symbiotic model, it is ethical. However, extraction without contribution is deeply unethical because when companies use open-source systems, repackage the code into managed cloud services, and contribute nothing back to the maintainers, they violate the unwritten social contract of open source, even if they comply with the legal license.

In software, "standing on the shoulders of giants" means that no application is built in a vacuum. When I write code for a university app or run a MediaPipe computer vision model, my code relies on Python, which relies on C libraries, which relies on system calls, which rely on the Linux Kernel, which relies on decades of computer science research. Open source makes this dependency tree visible and collaborative. It accelerates original innovation because developers do not have to waste time by starting from scratch and they can focus on the novel layers of their specific application.

Part B — Linux Footprint

The Linux Kernel is not an application you open, it is the environment in which everything else runs. To audit its footprint, we must look at where it initializes and how it manages the system.

- Installation Method: The kernel is typically installed via the distribution's package manager. On Ubuntu, running sudo apt install linux-image-generic pulls the latest compiled kernel. Users can also download the raw C source code from kernel.org and compile it manually using make.

- Key Directories:
    - /boot/: Contains the actual compiled kernel executable, usually named vmlinuz-[version]. It also holds the initrd.img (Initial RAM Disk) used during the boot process.
    - /lib/modules/: Contains loadable kernel modules (LKMs). The kernel is monolithic, but it uses modules for hardware drivers (like Wi-Fi cards or webcams) that can be loaded and unloaded dynamically without rebooting.
    - /proc/ and /sys/: Virtual filesystems created in RAM by the kernel to provide real-time hardware and process information to the user space.

- User and Group Security: The kernel does not run as a user—not even as root. It runs in "Ring 0" or kernel space, an isolated privilege level of the CPU. The root user operates in "Ring 3" (user space) and merely has the highest permissions granted *by* the kernel. This absolute separation prevents standard user applications from directly overwriting hardware memory.
- Service Management: The kernel itself is not managed like a standard service (you cannot systemctl restart linux). However, its modules can be managed dynamically using commands like lsmod (list modules), modprobe (add/remove modules), and sysctl (configure kernel parameters at runtime).

Update Model: Security patches are distributed via repositories (e.g., APT). Modern Linux systems also support livepatching (kpatch or Canonical Livepatch), which allows critical security updates to be injected directly into the running kernel's RAM without requiring a system reboot.

Part C — The FOSS Ecosystem

The Linux Kernel exists at the center of a massive open-source ecosystem. Without the surrounding tools, the kernel is just a silent engine with no steering wheel.

- Dependencies: The Linux Kernel cannot be compiled without GCC (GNU Compiler Collection). When Torvalds first wrote Linux, he specifically adapted it so it could be compiled by GCC, permanently binding the Linux kernel with the GNU toolchain.
- What it has enabled:
  - Android: The most widely used mobile operating system in the world is a direct fork of the Linux Kernel.
  - Containerization (Docker/Kubernetes): The kernel features cgroups (control groups) and namespaces, which allow for the isolation of resources. The entire modern cloud infrastructure is built on these kernel features.
- The LAMP Stack: Linux is the "L" in LAMP. While Apache serves the web pages, MySQL stores the data, and PHP processes the logic, the Linux kernel manages the TCP/IP networking stack, handles the disk I/O for the database, and schedules the CPU time for all of them.
- Community and Governance: The Linux Kernel is governed by the Linux Foundation. While Linus Torvalds remains the ultimate arbiter (the "Benevolent Dictator for Life"), the actual day-to-day development is highly decentralized across mailing lists (LKML - Linux Kernel Mailing List) where thousands of developers submit patches.

Part D — Open Source vs Proprietary

Dimension	Linux Kernel (Open Source)	Windows NT Kernel (Alternative)
Cost	Free, it has no costs.	Cost of Windows OS licenses (OEM or Retail).
Security (who can audit the code?)	Global researchers audit the code and patch zero-days rapidly.	Security relies entirely on Microsoft's internal engineering teams.
Support and reliability	Enterprise support available via Red Hat or Canonical.	Official guaranteed SLA support from Microsoft for enterprise clients.
Freedom to modify	GPL v2 allows anyone to rebuild, strip down, or customize the kernel.	Modification of the NT kernel violates Microsoft's EULA.

Community vs corporate control	Maintained by the community and the Linux Foundation, funded by multiple competing tech giants.	100% corporate control. Features and telemetry are controlled by Microsoft.

Verdict

The Linux Kernel is the best choice for server infrastructure, scientific computing, embedded systems, and mobile architecture. It can be changed a lot because it is open-source. It can be made smaller to work on a smart thermostat or bigger to work on an IBM supercomputer. The Windows NT kernel is very powerful, but it is closed, so users have to go along with Microsoft's design rules, telemetry, and upgrade cycles. Linux is the best choice for anything you want to do in the real world. I will keep using it for my own work. You need to know a lot about C programming to help with the kernel, but even just reporting bugs or helping with documentation is a great way to get involved in the ecosystem.

3. Shell Script Tasks

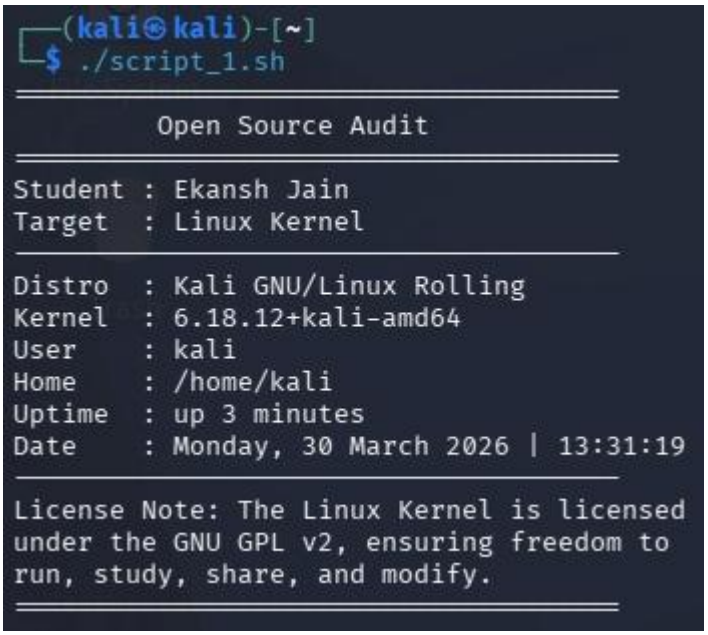
Script 1 — System Identity Report

This script utilizes command substitution ($()) to execute system commands and store their output dynamically. Variables are used to hold student and software data, and echo is used with formatting to display a clean, readable dashboard in the terminal.

Bash

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Ekansh Jain
# Course: Open Source Software
# Purpose: Displays a welcome dashboard with system details.

# --- Variables [cite: 105, 106, 107]
STUDENT_NAME="Ekansh Jain"
SOFTWARE_CHOICE="Linux Kernel"

# --- System info captured via command substitution [cite: 99, 110, 111]
# uname -r gets the kernel version [cite: 110]
KERNEL=$(uname -r)
# whoami captures the current logged-in user [cite: 110]
USER_NAME=$(whoami)
# uptime -p provides a "pretty" version of system uptime [cite: 111]
UPTIME=$(uptime -p)
# Captures the home directory dynamically for the current user [cite: 96]
USER_HOME=$HOME
# Extracts the OS name from the standard os-release file
DISTRO=$(grep '^PRETTY_NAME=' /etc/os-release | cut -d'=' -f2 | tr -d '"')
# Formats the current date and time [cite: 97]
DATE_TIME=$(date '+%A, %d %B %Y | %H:%M:%S')

# --- Display [cite: 113, 114, 117]
echo "======================================="
echo "        Open Source Audit            "
```

```
echo "========================================"
echo "Student : $STUDENT_NAME"
echo "Target  : $SOFTWARE_CHOICE"
echo "----------------------------------------"
echo "Distro  : $DISTRO"
echo "Kernel  : $KERNEL"
echo "User    : $USER_NAME"
echo "Home    : $USER_HOME"
echo "Uptime  : $UPTIME"
echo "Date    : $DATE_TIME"
echo "----------------------------------------"
# Message regarding the open-source license [cite: 98]
echo "License Note: The Linux Kernel is licensed"
echo "under the GNU GPL v2, ensuring freedom to"
echo "run, study, share, and modify."
echo "========================================"
```



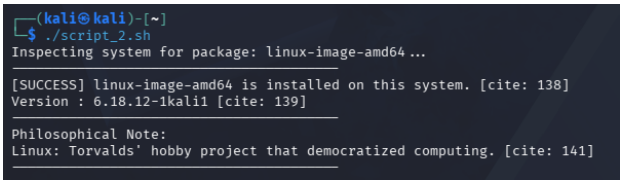
Script 2 — FOSS Package Inspector

This script employs defensive programming with an if-then-else statement to check package existence using dpkg -l. It suppresses standard output using &> /dev/null. It also utilizes a case statement for conditional branching based on the package name.

Bash

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: Ekansh Jain
# Course: Open Source Software
# Purpose: Checks if a package is installed and provides a philosophical note.
# For Kali Linux, the kernel metapackage is usually linux-image-amd64
PACKAGE="linux-image-amd64"
echo "Inspecting system for package: $PACKAGE..."
echo "----------------------------------------"
# Check if package is installed using dpkg -l
# Redirecting output to /dev/null for a clean 'silent' check
if dpkg -l "$PACKAGE" &> /dev/null; then
```

```
    echo "[SUCCESS] $PACKAGE is installed on this system." [cite: 138]
    # Extract version details using awk [cite: 148]
    # We use grep '^ii' to ensure we only look at 'installed' status lines
    VERSION=$(dpkg -l "$PACKAGE" | grep '^ii' | awk '{print $3}')
    echo "Version : $VERSION" [cite: 139]
else
    echo "[WARNING] $PACKAGE is NOT installed." [cite: 140]
    echo "Hint: You can install it using 'sudo apt install $PACKAGE'"
fi
echo "----------------------------------------"
echo "Philosophical Note:"
# Case statement for conditional branching [cite: 141]
case "$PACKAGE" in
    "linux-image-amd64" | "linux-image-generic" | "linux")
        echo "Linux: Torvalds' hobby project that democratized computing." [cite: 141]
        ;;
    "apache2" | "httpd")
        echo "Apache: The web server that built the open internet." [cite: 143]
        ;;
    "mysql-server" | "mariadb-server")
        echo "MySQL/MariaDB: Open source at the heart of millions of apps."
        ;;
    "gcc")
        echo "GCC: The compiler that built the open source world." [cite: 75]
        ;;
    "nmap")
        echo "Nmap: The 'Eyes of the Network' - essential for security auditing."
        ;;
    *)
        echo "A tool shared freely by the FOSS community." [cite: 10]
        ;;
esac
echo "----------------------------------------"
```



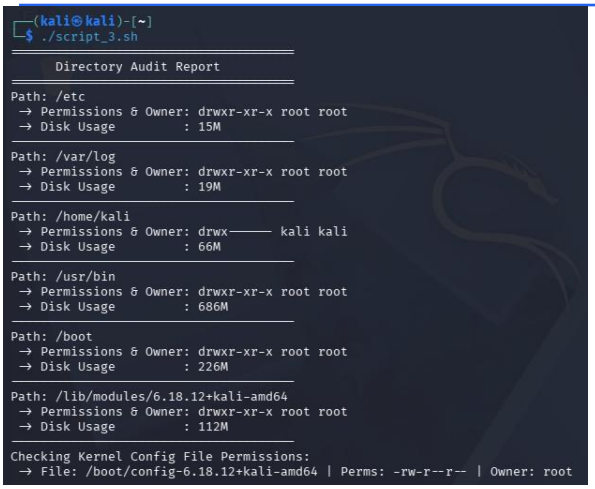
Script 3 — Disk and Permission Auditor

This script relies heavily on a for loop to iterate through a predefined bash array of critical system directories. It uses if [-d ...] to verify the directory exists, and pipes output through awk and cut to parse specific data columns (permissions and size).

Bash

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
```

```
# Author: Ekansh Jain
# Course: Open Source Software
# Purpose: Uses a for loop to audit permissions and disk usage of critical system paths.
# Array of critical system directories, including paths relevant to the Linux Kernel
# $HOME is used to dynamically find your current user's directory on Kali
DIRS=("/etc" "/var/log" "$HOME" "/usr/bin" "/boot" "/lib/modules/$(uname -r)")
echo "======================================"
echo "      Directory Audit Report          "
echo "======================================"
# Loop through the bash array using the ${DIRS[@]} syntax
for DIR in "${DIRS[@]}"; do
    # Check if the directory exists using the -d flag
    if [ -d "$DIR" ]; then
        # Pull permissions, owner, and group from ls using awk
        # $1=Permissions, $3=Owner, $4=Group
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')
        # Capture human-readable size using du and cut to extract the first field [cite: 148, 157]
        # 2>/dev/null hides 'Permission Denied' errors for sensitive sub-folders
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        echo "Path: $DIR"
        echo " -> Permissions & Owner: $PERMS"
        echo " -> Disk Usage         : $SIZE"
        echo "--------------------------------------"
    else
        echo "Path: $DIR"
        echo " -> [!] This directory does not exist on this system."
        echo "--------------------------------------"
    fi
done
# Section for software-specific config audit as per project requirements
KERNEL_CONFIG="/boot/config-$(uname -r)"
echo "Checking Kernel Config File Permissions:"
if [ -f "$KERNEL_CONFIG" ]; then
    ls -l "$KERNEL_CONFIG" | awk '{print " -> File: "$9" | Perms: "$1" | Owner: "$3}'
else
    echo " -> [!] Kernel config not found in /boot."
fi
echo "======================================"
```



Script 4 — Log File Analyzer

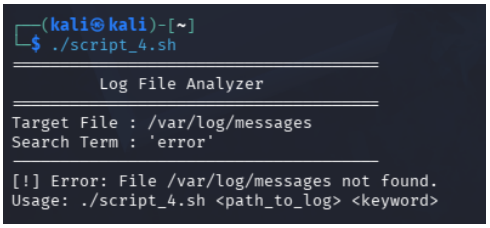
This script accepts command-line arguments ($1, $2), establishing defaults if arguments are missing. The core mechanism is a while IFS= read -r loop, which is the safest way to read files line-by-line in bash. It uses arithmetic expansion $((...)) to maintain a counter variable.

Concepts to use: while read loop, if-then, counter variables, command-line arguments ($1).

Bash

```
#!/bin/bash
# Script 4: Log File Analyzer
# Author: Ekansh Jain
# Course: Open Source Software
# Purpose: Analyzes log files line-by-line using a while-read loop and counters.
# Usage: ./script4.sh /var/log/messages error

# --- Command line arguments with defaults
# $1: The log file to scan (Default: /var/log/messages for Kali/Debian)
# $2: The keyword to search for (Default: 'error')
LOGFILE=${1:-"/var/log/messages"}
KEYWORD=${2:-"error"}
COUNT=0

echo "========================================"
echo "        Log File Analyzer            "
echo "========================================"
echo "Target File : $LOGFILE"
echo "Search Term : '$KEYWORD'"
echo "----------------------------------------"

# --- File Validation
# Check if the file exists and is a regular file
if [ ! -f "$LOGFILE" ]; then
    echo "[!] Error: File $LOGFILE not found."
    echo "Usage: $0 <path_to_log> <keyword>"
    exit 1
fi

# Check if the file is empty using -s (size greater than zero)
if [ ! -s "$LOGFILE" ]; then
```

```
    echo "[!] Note: $LOGFILE exists but is currently empty."
    exit 0
fi

# --- Core Logic: Safe Line-by-Line Reading
# IFS= prevents leading/trailing whitespace from being trimmed
# -r prevents backslash escapes from being interpreted
while IFS= read -r LINE; do
    # Check if the line contains the keyword (case-insensitive)
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        # Arithmetic expansion for the counter
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "Result: Found '$KEYWORD' $COUNT times."
echo ""
echo "Last 5 matching occurrences:"
echo "-------------------------------------"
# Display the last 5 matching lines using tail as per project TODO
grep -i "$KEYWORD" "$LOGFILE" | tail -n 5
echo "======================================"
```



Script 5 — The Open Source Manifesto Generator

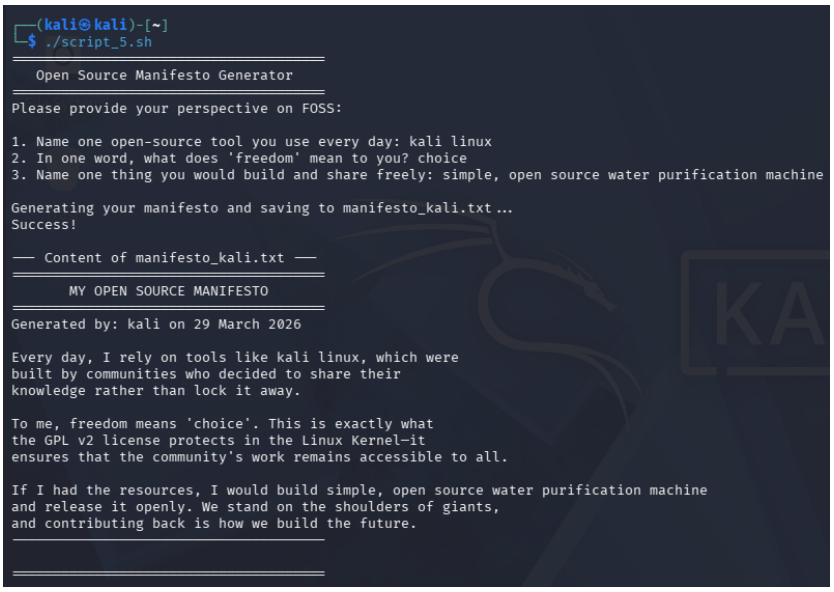
This script emphasizes interactivity using the read -p command to gather user input. It groups output commands using { ... } > file.txt to redirect a block of text into a newly generated file dynamically named using the $(whoami) command.

Concepts to use: read for user input, string concatenation, writing to a file with >, date command, aliases concept demonstrated via a comment.

Bash

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: Ekansh Jain
# Course: Open Source Software
# Purpose: Interactive script to generate a personalized FOSS manifesto file.

# --- Concept: Aliases (Unit 5)
# Note: In a live shell, one might use: alias today='date +"%d %B %Y"'
# For this script's reliability across different shells, we use a variable.
DATE_STRING=$(date '+%d %B %Y')
USER_ID=$(whoami)
OUTPUT_FILE="manifesto_${USER_ID}.txt"

echo "======================================"
echo "   Open Source Manifesto Generator    "
echo "======================================"
echo "Please provide your perspective on FOSS:"
echo ""
```

```
# --- Gather user input interactively using read -p
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

echo ""
echo "Generating your manifesto and saving to $OUTPUT_FILE..."

# --- Grouping commands using { ... } and redirecting output to a file
# This demonstrates block redirection (>) using a dynamically named file
{
   echo "========================================"
   echo "      MY OPEN SOURCE MANIFESTO      "
   echo "========================================"
   echo "Generated by: $USER_ID on $DATE_STRING"
   echo ""
   echo "Every day, I rely on tools like $TOOL, which were"
   echo "built by communities who decided to share their"
   echo "knowledge rather than lock it away."
   echo ""
   echo "To me, freedom means '$FREEDOM'. This is exactly what"
   echo "the GPL v2 license protects in the Linux Kernel—it"
   echo "ensures that the community's work remains accessible to all."
   echo ""
   echo "If I had the resources, I would build $BUILD"
   echo "and release it openly. We stand on the shoulders of giants,"
   echo "and contributing back is how we build the future."
   echo "----------------------------------------"
} > "$OUTPUT_FILE"

echo "Success!"
echo ""
echo "--- Content of $OUTPUT_FILE ---"
# Display the file content to the terminal using cat
cat "$OUTPUT_FILE"
echo ""
echo "========================================"
```



4. References

- Linux Kernel Archives: https://www.kernel.org/category/about.html
- The Linux Foundation: https://www.linuxfoundation.org/
- GNU General Public License v2.0: https://www.gnu.org/licenses/old-licenses/gpl-2.0.html
- Debian Package Management Docs: https://www.debian.org/doc/manuals/debian-faq/pkg-basics.en.html
- Bash Scripting Guide: https://www.gnu.org/software/bash/manual/bash.html
- Linux Man Pages (Project Docs): https://man7.org/linux/man-pages/
- FOSS Philosophy (Free Software Foundation): https://www.gnu.org/philosophy/free-sw.html

5. Conclusion

The Linux Kernel is the best example of how powerful the open-source philosophy is. The GPL v2 license protects and drives it, and it started as a student's desire for an easy-to-use operating system. Now it is the foundation of the modern digital world. The present audit demonstrates that Linux surpasses proprietary alternatives, such as the Windows NT kernel, in terms of security, adaptability, and openness, while simultaneously fostering a vast, collaborative environment. The practice of open development transcends mere licensing considerations; it embodies a commitment to collective improvement. Consequently, I advocate for the practical application of Linux, as comprehension and utilization of its architecture constitute a crucial component of contemporary software professional competence.